

Midsem Solutions

Lecture 14

March 9, 2026

Q1

(a) A new system, called SMILE, in the CSE department of IIT Kanpur has 40 nodes. Each node has 10 cores. This means that a job executed on SMILE via the command **mpiexec -np 500 -f hostfile ./executable** will fail, because of fewer available resources. Assume *hostfile* has the names of all the nodes of SMILE.

FALSE

Oversubscription

(FYI: OS generally has a very high limit ($> 10^5$) on the maximum number of processes, system-wide and per user)

Recap (Lecture 10)

64 processes on 2 nodes, ppn=16

Hello from rank 0 out of 64 on system rdcn26

Hello from rank 1 out of 64 on system rdcn26

....

Hello from rank 14 out of 64 on system rdcn26

Hello from rank 15 out of 64 on system rdcn26

Hello from rank 16 out of 64 on system rdcn27

Hello from rank 17 out of 64 on system rdcn27

....

Hello from rank 30 out of 64 on system rdcn27

Hello from rank 31 out of 64 on system rdcn27

Hello from rank 32 out of 64 on system rdcn26

Hello from rank 33 out of 64 on system rdcn26

....

Hello from rank 46 out of 64 on system rdcn26

Hello from rank 47 out of 64 on system rdcn26

Hello from rank 48 out of 64 on system rdcn27

Hello from rank 49 out of 64 on system rdcn27

....

Hello from rank 62 out of 64 on system rdcn27

Hello from rank 63 out of 64 on system rdcn27

Example of oversubscription on PARAM Rudra

Q1

(b) If a user submits an MPI job to the above-mentioned system, SMILE, the user must first install a job scheduler on SMILE, otherwise the user's job will not run on SMILE.

FALSE

Recap (Lecture 3)

Same host

- `mpirun -np 6 ./program.x`

Multiple hosts

- `mpirun -np 6 -f hostfile ./program.x`
 - Round robin process placement
 - `mpirun -np 6 -hosts csews1,csews2 ./program.x`
 - Sequential/contiguous placement
 - `mpirun -np 6 -hosts csews1:3,csews2:3 ./program.x`

Recap (Lecture 3)

Beowulf PC Clusters: Breaking the Cost Barrier to High End Application Computing

Thomas Sterling, PhD

"Leader of NASA BEOWULF Project"

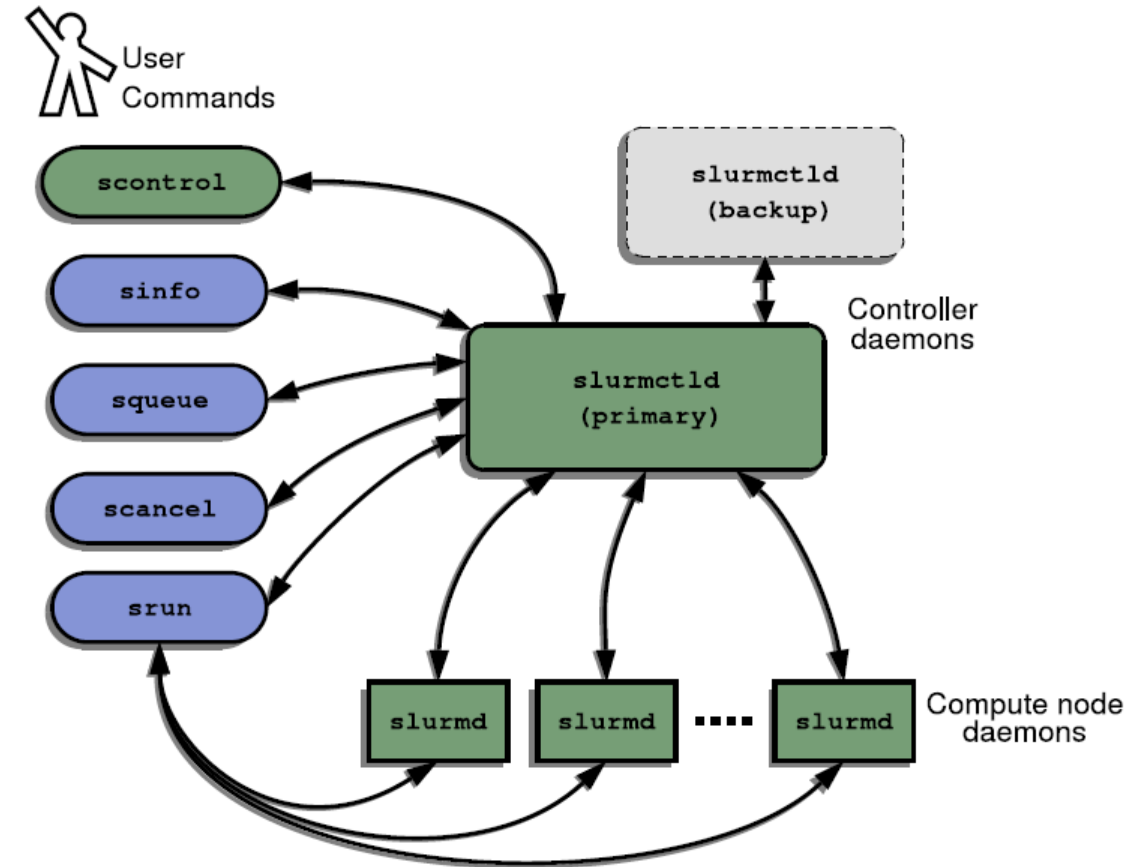
Senior Staff Scientist; HPC Systems Group, Jet Propulsion Lab
Visiting Professor, California Institute of Technology

This exciting presentation emphasizes the economical opportunity of networking low cost personal computers (PCs) to solve science and engineering problems. Such PC networks permit parallel or distributed computing in an effective and efficient manner. Various high performance computing applications using personal computer networks are addressed in terms of science performed and performance achieved. Performance metrics discussed include speedup vs. costs vs. scalability. Such experience has evolved with Dr. Sterling's NASA BEOWULF project and associated BEOWULF-class computing. Come and discuss your applications using a PC network for high performance computation!

Recap (Lecture 7)

Workload managers/Schedulers

- Portable Batch System (PBS)
- LoadLeveler
- Application Level Placement Scheduler (ALPS)
- Moab/Torque
- Simple Linux Utility for Resource Management (SLURM)



Q1

(c) The above-mentioned system, SMILE, has been designed to run at a peak throughput of 10 GFLOPS (floating point operations per second). This means that

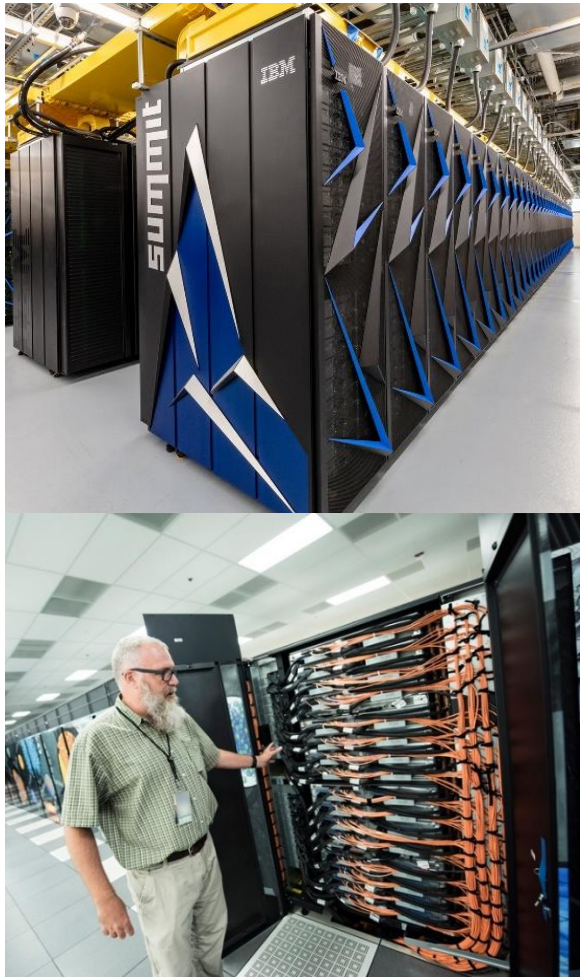
A job that runs on SMILE will take a maximum of 10 ns to run FALSE

Any job that runs on SMILE must run at 10 GFLOPS FALSE

No job will run at more than 10 GFLOPS TRUE

It is possible that a code that runs on SMILE and gives a throughput of less than 10 GFLOPS cannot be optimized further TRUE

Recap (Lecture 1) top500



Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

Recap (Lecture 1)

[HOME](#)[COMPLETE RESULTS](#)[GREEN GRAPH500](#)[SUBMISSIONS](#)[BE](#)

RANK ↕	PREVIOUS RANK ↕	MACHINE ↕	VENDOR ↕	TYPE ↕	NETWORK ↕	INSTALLATION SITE ↕	LOCATION ↕
1	new	Supercomputer Fugaku	Fujitsu	Fujitsu A64FX	Tofu Interconnect D	RIKEN Center for Computational Science (R-CCS)	Kobe Hyogo
2	new	Wuhan Supercomputer	HUST	Kunpeng 920+Tesla A100	Custom	Wuhan Supercomputing Center	Wuhan
3	3	Frontier	HPE	HPE Cray EX235a	Slingshot-11	DOE/SC/Oak Ridge National Laboratory	Oak Ridge TN
4	new	Pengcheng Cloudbrain-II	HUST-Pengcheng Lab-HUAWEI	Kunpeng 920+Ascend 910	Custom	Pengcheng Lab	ShenZhen

Ranked based on a different application set

Q1

(d) If a user submits a job on a supercomputer SMILE++ for a wall-clock time of 20 minutes, and the actual runtime of the user's job is 30 minutes, this means that

- The job will be terminated by the system administrator **FALSE**
- The job will be terminated by the scheduler **TRUE**
- The job will be in an idle state unless the user kills the job **FALSE**
- The user's job will run for 30 minutes; however, 10 minutes will be deducted from the next job of the user **FALSE**

Recap (Lecture 7)

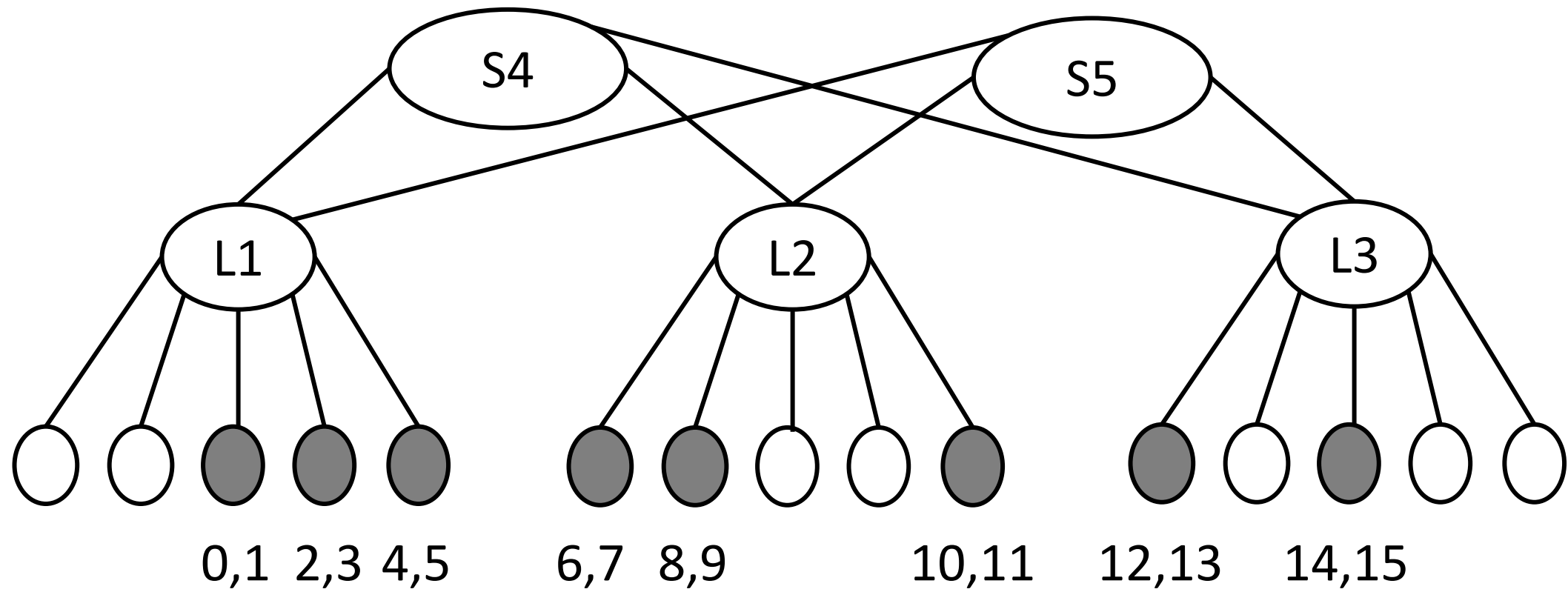
SLURM Commands

```
#SBATCH -N 2
#SBATCH --ntasks-per-node=16
#SBATCH --error=job.%J.err
#SBATCH --output=job.%J.out
#SBATCH --time=00:10:00
#SBATCH --partition=standard
mpirun -np 32 ./exe <args>
```

Job Script

```
sbatch <jobscript>
squeue
scancel <jobid>
```

Q2



2(a)

Derive the number of hops for *sender ranks 0 and 6*, for their *respective D1 and D2 communications*.

Rank 0: D1: Rank 4 (on the same leaf switch): 0 -> L1 -> 4 (2 hops)

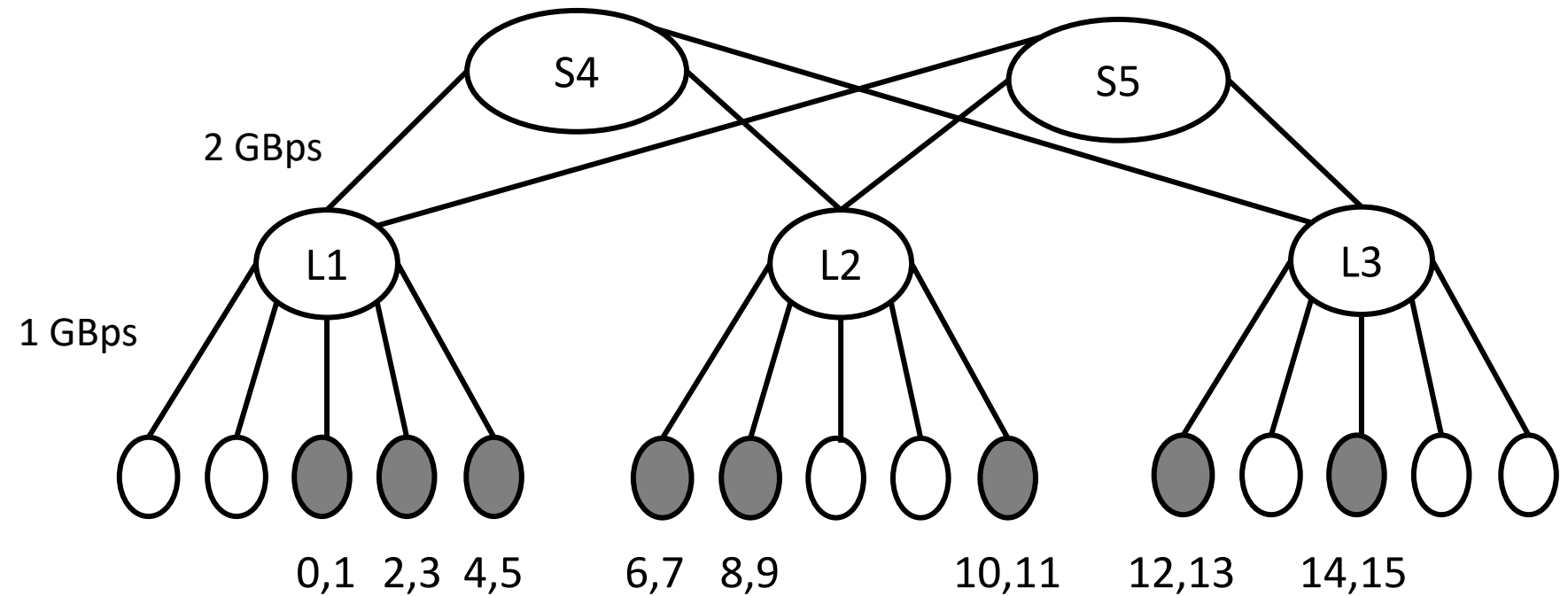
Rank 0: D2: Rank 8 (on a different leaf switch): 0 -> L1 -> S4/S5 -> L2 -> 8 (4 hops)

Rank 6: D1: Rank 10 (on the same leaf switch): 6 -> L2 -> 10 (2 hops)

Rank 6: D2: Rank 14 (on a different leaf switch): 6 -> L2 -> S4/S5 -> L3 -> 14 (4 hops)

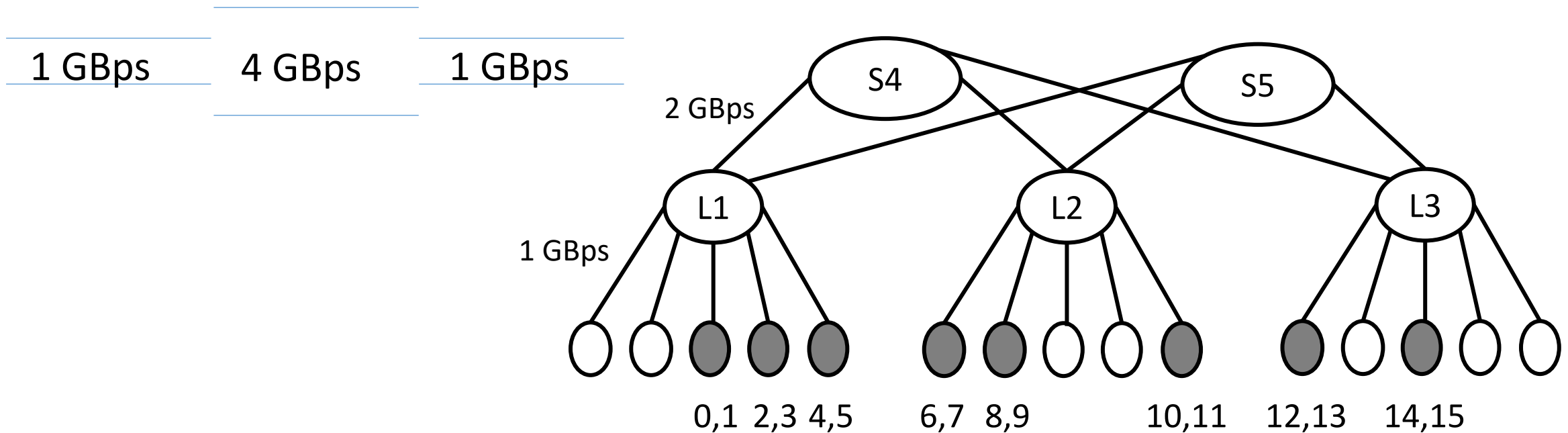
2(b)

Derive the maximum available bandwidth for sender ranks 0 and 6, for their respective D1 and D2 communications



2(b)

Derive the maximum available bandwidth for sender ranks 0 and 6, for their respective D1 and D2 communications



2(b)

Derive the maximum available bandwidth for sender ranks 0 and 6, for their respective D1 and D2 communications

Rank 0: D1: 0 → L1 → 4

1 Gbps, 1 Gbps

Rank 0: D2: 0 → L1 → S4/S5 → L2 → 8

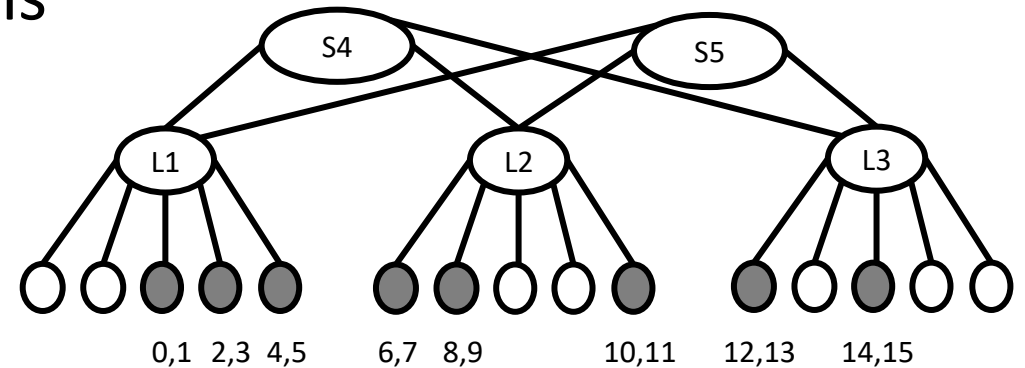
1 Gbps, 4 Gbps, 4 Gbps, 1 Gbps

Rank 6: D1: 6 → L2 → 10

1 Gbps, 1 Gbps

Rank 6: D2: 6 → L1 → S4/S5 → L3 → 14

1 Gbps, 4 Gbps, 4 Gbps, 1 Gbps

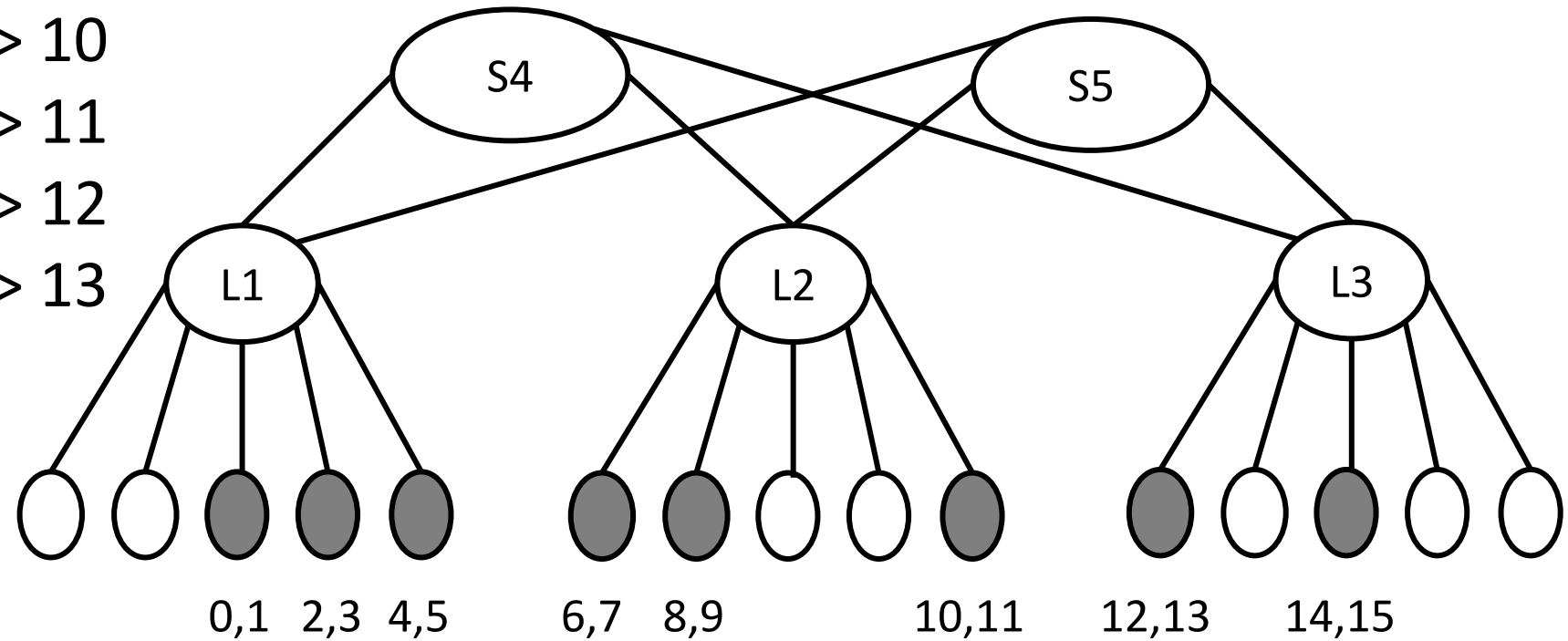


2(c)

Derive the total number of *outgoing communications to higher levels* (i.e. outward to spine switches) along with the *communication volumes* from L1 and L2 leaf switches for the given execution configuration for *sending to rank+D1 and rank+D2 ranks only*.

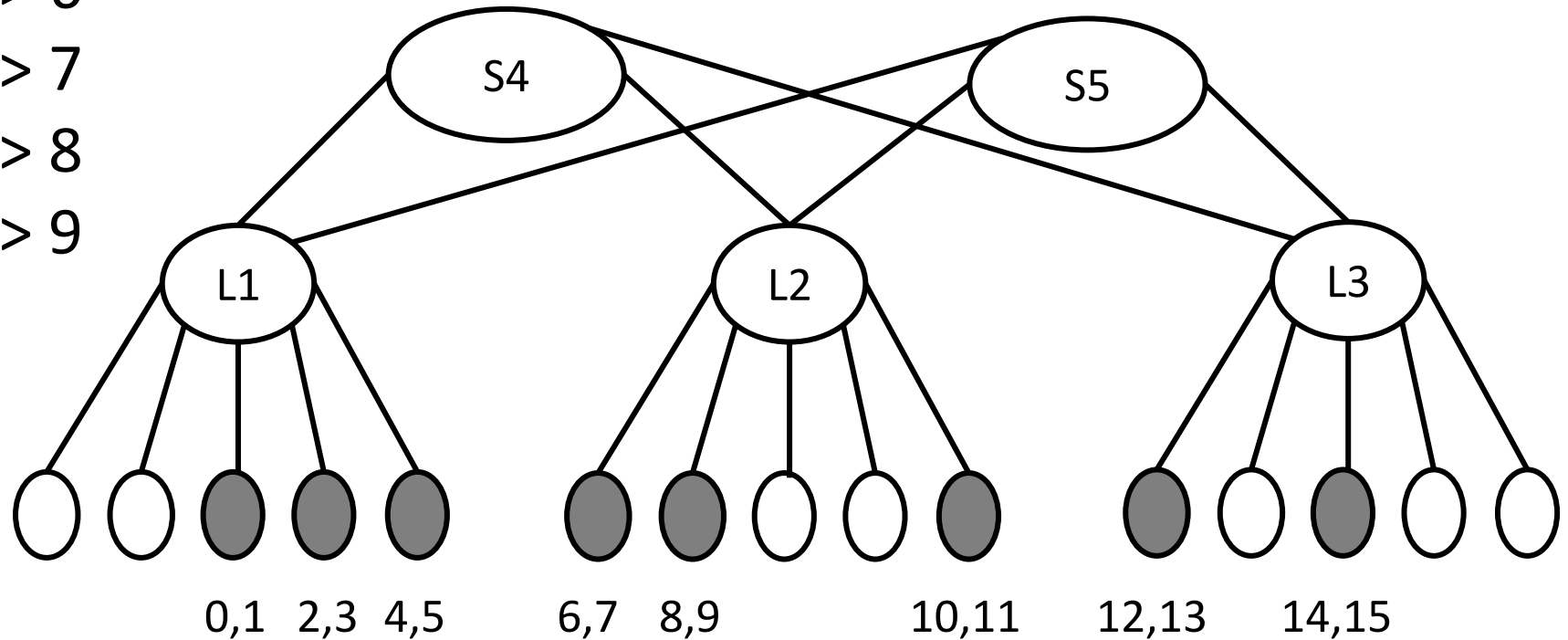
L1

0 -> L1 -> S4/S5 -> L2 -> 8
1 -> L1 -> S4/S5 -> L2 -> 9
2 -> L1 -> S4/S5 -> L2 -> 10
3 -> L1 -> S4/S5 -> L2 -> 11
4 -> L1 -> S4/S5 -> L3 -> 12
5 -> L1 -> S4/S5 -> L3 -> 13



L1

2 -> L1 -> S4/S5 -> L2 -> 6
3 -> L1 -> S4/S5 -> L2 -> 7
4 -> L1 -> S4/S5 -> L3 -> 8
5 -> L1 -> S4/S5 -> L3 -> 9

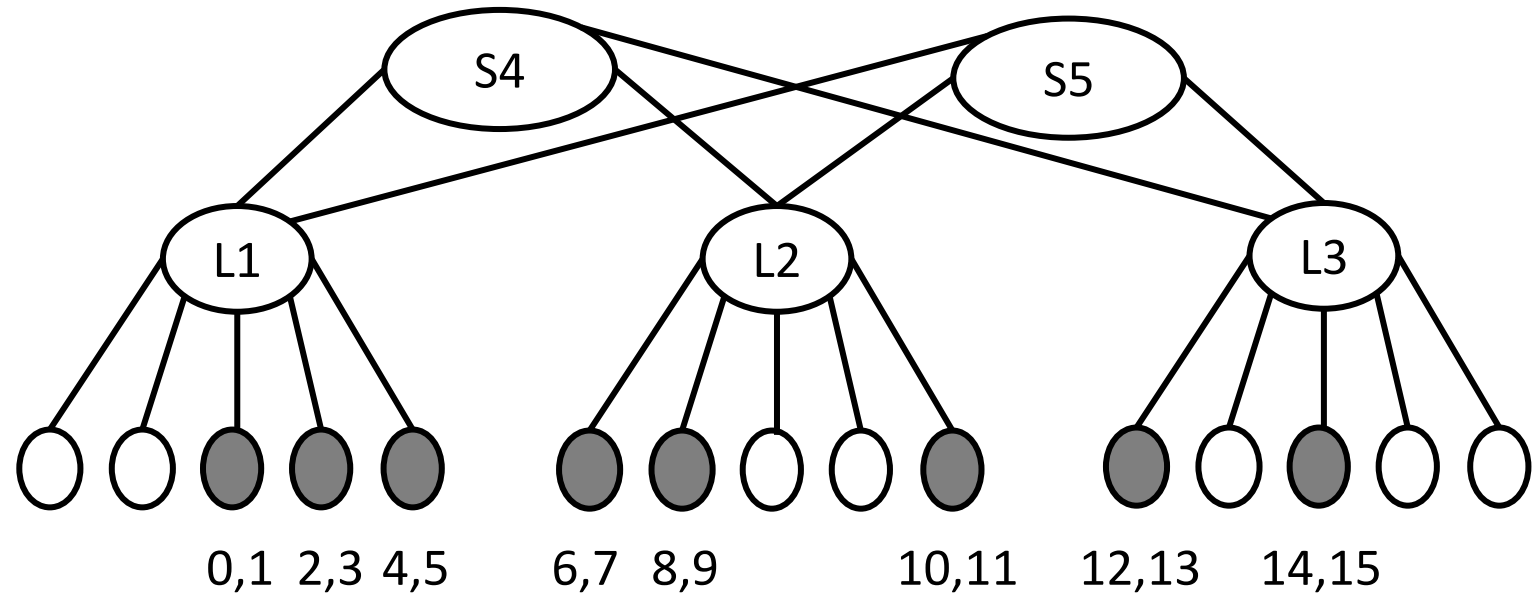


Thus, the total number of outgoing communications from L1 = 10

Communication volume = $10 * 1 \text{ MB} = 10 \text{ MB}$

L2

6 -> L2 -> S4/S5 -> L2 -> 14
7 -> L2 -> S4/S5 -> L2 -> 15
8 -> L2 -> S4/S5 -> L2 -> 12
9 -> L2 -> S4/S5 -> L2 -> 13
10 -> L2 -> S4/S5 -> L3 -> 14
11 -> L2 -> S4/S5 -> L3 -> 15



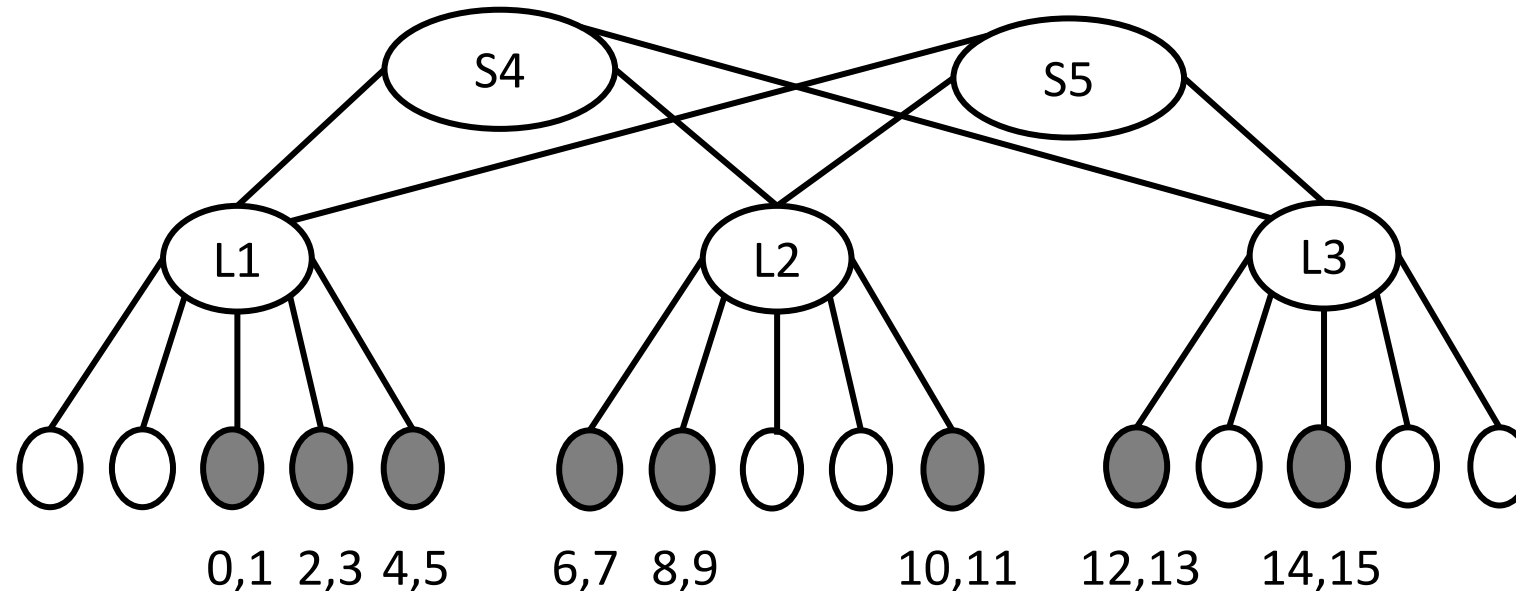
Thus, the total number of outgoing communications from L2 = 6

Communication volume = 6 * 1 MB = 6 MB

Importance

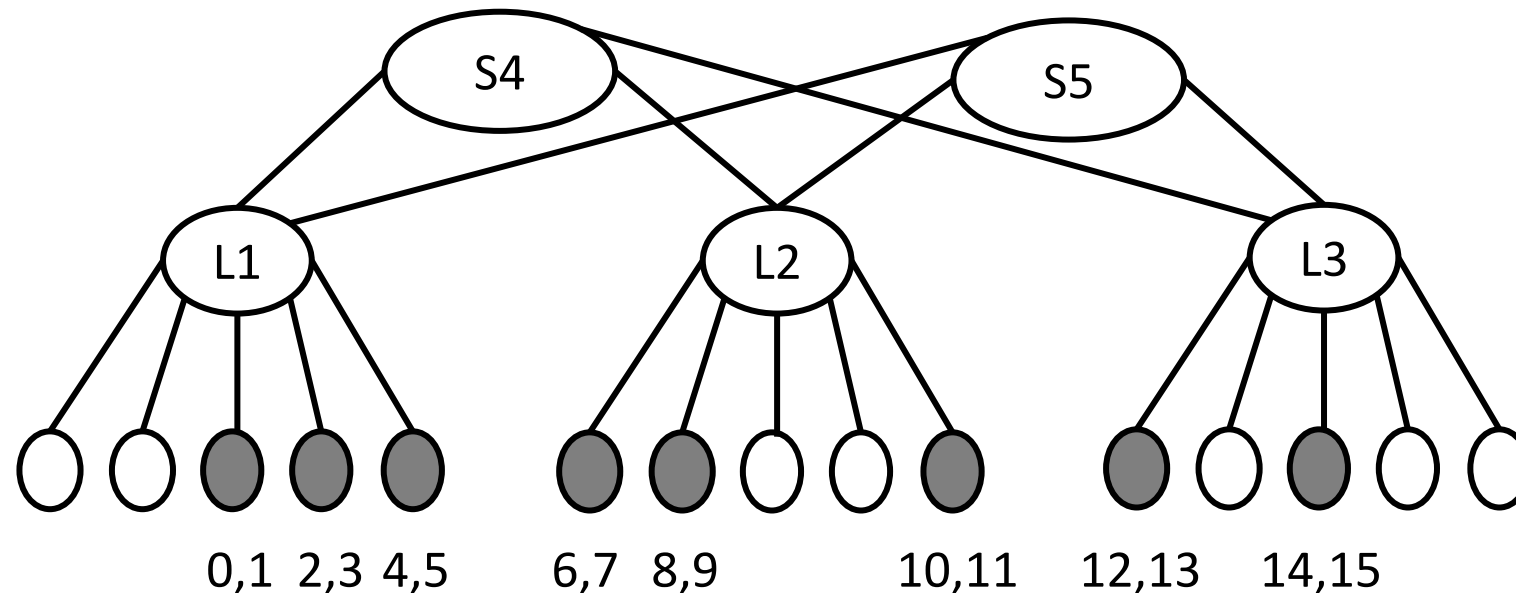
The total number of outgoing communications and the communication volume determine the load on the switch

Classwork: What is the total communication volume via L1 for naïve parallel sum algorithm (rank 0 is the root) for summing up 10^6 doubles per rank (1 double = 8 bytes)?



Homework

What is the total communication volume via L1 for optimized parallel sum algorithm (rank 0 is the root) for summing up 10^6 doubles per rank (1 double = 8 bytes)?



```
int MPI_Bcast (buffer, count,  
datatype, root, comm)
```

Q3: mpirun -np 4 ./exe 2 3

```
6    int myrank, size, color[]={17,27};  
7    int rank = atoi(argv[1]), number=atoi(argv[2]);  
13   MPI_Bcast (color, 2, MPI_INT, rank, MPI_COMM_WORLD);  
14   printf ("%d before %d %d\n", myrank, color[0], color[1]);  
16   if (myrank == rank) }  
17       color[0]=7;      }  
19   color[1] += number;  
21   MPI_Bcast (&color[1], 1, MPI_INT, rank, MPI_COMM_WORLD);  
22   printf ("%d after %d %d\n", myrank, color[0], color[1]);
```


mpirun -np 4 ./exe 2 3

0 before 17 27

1 before 17 27

2 before 17 27

3 before 17 27

0 after 17 30

1 after 17 30

2 after 7 30

3 after 17 30

Classwork

```
6      int myrank, size, color[]={17,27};
7      int rank = atoi(argv[1]), number=atoi(argv[2]);
13     MPI_Bcast (color, 2, MPI_INT, rank, MPI_COMM_WORLD);
14     printf ("%d before %d %d\n", myrank, color[0], color[1]);
16     if (myrank == rank)
17         color[0]=7;
19     color[1] += number;
21     MPI_Bcast (&color[0], 1, MPI_INT, rank, MPI_COMM_WORLD);
22     printf ("%d after %d %d\n", myrank, color[0], color[1]);
```

mpirun -np 4 ./exe 2 3

0 before 17 27

1 before 17 27

2 before 17 27

3 before 17 27

0 after 7 30

1 after 7 30

2 after 7 30

3 after 7 30

Q4

```
15  rows = atoi(argv[1]); cols = numtasks/rows;
16  if (numtasks%cols !=0 || numtasks%rows !=0)
MPI_Abort(MPI_COMM_WORLD, 0);
20  int color = myrank/cols;
21  color = myrank%cols < cols-1 ? color : rows+1;
22  MPI_Comm_split(MPI_COMM_WORLD, color, myrank, &new_comm);
23  MPI_Comm_rank (new_comm, &new_rank);
24  MPI_Comm_size (new_comm, &new_size);
```

Solving...

- `int color = myrank/cols;`

0 0 0

1 1 1

2 2 2

- `color = myrank%cols < cols-1 ? color : rows+1;`

0 0 4

1 1 4

2 2 4

Solving...

```
22 MPI_Comm_split(MPI_COMM_WORLD, color, myrank, &new_comm);  
23 MPI_Comm_rank (new_comm, &new_rank);  
24 MPI_Comm_size (new_comm, &new_size);
```

color	New_rank	myrank
0 0 4	0 1 0	0 1 2
1 1 4	0 1 1	3 4 5
2 2 4	0 1 2	6 7 8

Q4 continued

```
27 MPI_Gather (&myrank, 1, MPI_INT, all, 1, MPI_INT, 0, new_comm);
28
29 int sum=0;
30 if (new_rank == 0) {
31     for (int i=0; i<new_size; i++)
32         sum += all[i];
33 }
34 printf ("%d %d %d %d\n", myrank, color, sum, new_rank);
35
```

Q4(a)

```
mpirun -np 9 -hosts host1,host2,host3 ./exe 3 | sort -k1n
```

```
0 0 1 0
```

```
1 0 0 1
```

```
2 4 15 0
```

```
3 1 7 0
```

```
4 1 0 1
```

```
5 4 0 1
```

```
6 2 13 0
```

```
7 2 0 1
```

```
8 4 0 2
```


Q4(b)

(i) Mention the rank placement on the three hosts

Host1 0, 3, 6

Host2 1, 4, 7

Host3 2, 5, 8

Q4(b)

(ii) For a naïve many-to-one implementation of MPI_Gather (line 27), derive the average number of hops, assuming the hosts connectivity is as shown above. *Note 0, 3, 6, and 2 are the 0th rank processes in new_comm.*

Hops from 0 to 0 = 0 Hops from 6 to 6 = 0 Hops from 2 to 2 = 0

Hops from 1 to 0 = 1 Hops from 7 to 6 = 1 Hops from 5 to 2 = 0

Hops from 8 to 2 = 0

Hops from 3 to 3 = 0

Hops from 4 to 3 = 1

The average number of hops: 3/9

Q5

```
12 color = rank%4;
13 MPI_Comm_split (MPI_COMM_WORLD, color, rank, &newcomm);
14 MPI_Comm_rank (newcomm, &newrank);
15 MPI_Comm_size (newcomm, &newsiz);
16
17 MPI_Allreduce (&newrank, &maxrank, 1, MPI_INT, MPI_MAX, newcomm);
18 if (!newrank) printf ("%d New %d Max %d\n", rank, newrank, maxrank);
19
20 MPI_Allreduce (&rank, &maxrank, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
21 if (!rank) printf ("%d Max %d\n", rank, maxrank);
```

mpirun -np 84 ./exe | sort -k1n

```
17 MPI_Allreduce (&newrank, &maxrank, 1, MPI_INT, MPI_MAX, newcomm);  
18 if (!newrank) printf ("%d New %d Max %d\n", rank, newrank, maxrank);
```

0 New 0 Max 20

1 New 0 Max 20

2 New 0 Max 20

3 New 0 Max 20

```
mpirun -np 84 ./exe | sort -k1n
```

```
20  MPI_Allreduce (&rank, &maxrank, 1, MPI_INT, MPI_MAX,  
MPI_COMM_WORLD);
```

```
21  if (!rank) printf ("%d Max %d\n", rank, maxrank);
```

```
0 Max 83
```

Q6

```
13  if (myrank%2 == 0)
14      MPI_Send (arr, numElements, MPI_BYTE, myrank+1, myrank,
MPI_COMM_WORLD);
15  else
16      MPI_Recv (recvarr, numElements, MPI_BYTE, myrank-1,
myrank-1, MPI_COMM_WORLD, &status);
```

Q6(a)

(A) `time mpirun -np 60 --hosts host1,host2,host3,host4 ./3 100`

(B) `time mpirun -np 60 --hosts host1,host2,host3,host4 ./3 10000000`

(C) `time mpirun -np 120 --hosts host1,host2,host3,host4 ./3 10000000`

2.8s

1.4s

0.2s

C

B

A

Q6(b)

(i) `mpirun -np 120 --hosts host1,host2,host3,host4 ./3 6000000`

(ii) `mpirun -np 120 --hosts host1:30,host2:30,host3:30,host4:30 ./3 6000000`

(i) Takes longer to execute due to several simultaneous internode communications among same hosts, for e.g. ranks 0 and 1 communicate at the same time as ranks 4 and 5, and ... on the link between host1 and host2,

All communications in (ii) are intra-node.

Q6(c)

Assume 12-core hosts. Execution command is **mpirun -np 24 --hosts host1:6,host2:6,host3:6,host4:6 ./exe 1024** Assume that host1 – host4 are connected using a linear array, i.e. number of hops from host1 to host4 = 3. Assume that the above execution took 0.1 seconds. Derive the communication bandwidth achieved by the code in this case.

There are 12 communications, each of 1024 bytes.

Therefore, total data communicated in 0.1 s = $12 * 1024$ bytes = 12 KB

Thus, the communication bandwidth achieved = $12 \text{ KB} / 0.1 \text{ s} = 120 \text{ KBps}$

Q7

```
25  time1 = MPI_Wtime ();
26
27  for (i=local*rank; i<local*(rank+1); i++)
28    sumLocal += array[i] * array[i];
29
30  if (numprocs == 1)
31    sumGlobal = sumLocal;
32  else
33    MPI_Reduce (&sumLocal, &sumGlobal, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
34
```

Q7

max_time should be
initialized to 0.0

```
35 time1 = MPI_Wtime () - time1;
36 MPI_Reduce (&time1, &max_time1, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
37
38 int color = rank%number; MPI_Comm comm;
39 MPI_Comm_split (MPI_COMM_WORLD, color, rank, &comm);
40 MPI_Reduce (&time1, &max_time, 1, MPI_DOUBLE, MPI_MAX, 0, comm);
41 MPI_Reduce (&max_time, &max_time2, 1, MPI_DOUBLE, MPI_MAX, 0,
MPI_COMM_WORLD);
42
43 if (!rank) printf ("%lf %lf\n", max_time1, max_time2);
```

2-step reduce,
achieving the
same result

Q7(a)

(A) `mpirun -np 120 -hosts host1:40,host2:40,host3:40 ./5 600 4`

(B) `mpirun -np 120 -hosts host1:40,host2:40,host3:40 ./5 600 16`

0.050770 0.050770

0.109982 0.109982

Q7(b)

```
mpirun -np 32 -hosts host1:8,host2:8,host3:8,host4:8 ./exe 6000000 16
```

```
max_time1 -> 0.0035
```

```
mpirun -np 128 -hosts host1:32,host2:32,host3:32,host4:32 ./exe  
6000000 16
```

```
max_time1 -> 0.0085
```

Q8

```
16  for (int i=0; i<size; i++)
17  for (int j=0; j<count; j++) {
18      data[i][j] = 0;
19      if (i == rank) data[i][j] = rank+j;

22  for (int i=0; i<size; i++)
23      MPI_Bcast (&data[i][0], count, MPI_INT, i, MPI_COMM_WORLD);

25  for (int i=0; i<size; i++) {
26      printf("%d ", rank);
27      for (int j=0; j<count; j++) {
28          printf ("%d ", data[i][j]);    // replace data with recvddata for Q8-a.
29      }
30      printf("\n");
31  }
```

```
mpirun -np 3 ./exe 3
```

```
0 0 1 2
```

```
0 1 2 3
```

```
0 2 3 4
```

```
1 0 1 2
```

```
1 1 2 3
```

```
1 2 3 4
```

```
2 0 1 2
```

```
2 1 2 3
```

```
2 2 3 4
```

Q9

```
12 data1 = rank/2+rank;
13 data2 = rank/2+2*rank;
14
15 if (rank < size/2) { // sends
16     printf ("%d %d %d %d %d\n", rank, rank+size/2, size-1-rank, data1, data2);
17     MPI_Send (&data1, 1, MPI_INT, rank+size/2, rank, MPI_COMM_WORLD);
18     MPI_Send (&data2, 1, MPI_INT, size-1-rank, rank, MPI_COMM_WORLD);
19 }
20 else { // receives
21     printf ("%d %d %d %d %d\n", rank, rank-size/2, size-1-rank, data1, data2);
22     MPI_Recv (&data1, 1, MPI_INT, rank-size/2, rank-size/2, MPI_COMM_WORLD, &status);
23     MPI_Recv (&data2, 1, MPI_INT, size-1-rank, size-1-rank, MPI_COMM_WORLD, &status);
24 }
```


Q9(a)

0 2 3 0 0

1 3 2 1 2

2 0 1 3 5

3 1 0 4 7

Q10

Derive the communication-to-computation ratio in the case of 9-point stencil computations (example shown in the figure) for two cases (a) and (b) as mentioned below. Each grid point value is computed from its 8 neighbors (as shown by the unfilled circles in the figure – 2 each in north, south, east, and west directions). Assume that there are $P \times P$ processes and the 2D grid size is $N \times N$. (a) Assume a 1D domain decomposition. (b) Assume a 2D domain decomposition. Illustrate the decompositions with the help of small diagrams.

(a) Communications = $2 * 2N$, Computations = N^2/P^2

Ratio = $4P^2/N$

(b) Communications = $4 * 2 * N/P$, Computations = N^2/P^2

Ratio = $8P/N$